Experiment 6

Student Name: Ashutosh Yadav **UID: 23BCS11023**
Branch: BE CSE **Section/Group: KRG-3-A**
Semester: 6th **Date of Performance: 02/03/26**
Subject Name: Full Stack Development **Subject Code: 23CSH-309**

Aim:
To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

Objective:
After completing this experiment, students will be able to:

1. Understand Spring Boot fundamentals
   - Learn the role of Spring Boot in backend development.
   - Configure and run a basic Spring Boot application.

2. Apply IoC and Dependency Injection - Understand how Spring manages object creation and dependencies.
   - Use annotations like @Autowired, @Service, and @Repository.

3. Implement Layered Architecture
   - Structure the application using Controller, Service, and Repository layers.
   - Understand separation of concerns in backend design.

4. Develop RESTful APIs

- Create CRUD operations using Spring Boot REST controllers.
- Design APIs using proper HTTP methods (GET, POST, PUT, DELETE).

5. Use DTOs for API communication - Implement DTO mapping to separate internal models from API responses.

6. Implement Validation
   - Apply validation using Jakarta Validation annotations (@NotNull, @Email, etc.).

7. Handle Exceptions Globally
   - Implement global exception handling using @ControllerAdvice.

8. Build the HealthHub Backend Module
   - Develop backend services for patient/health record management.

<u>Implementation/Code</u>:

PatientController.java:

```
package com.healthhub.controller;

import java.util.List;

import
org.springframework.beans.factory.annotation.A
utowired;
import
org.springframework.web.bind.annotation.*;

import com.healthhub.model.Patient;
import com.healthhub.service.PatientService;
```

```
@RestController
@RequestMapping("/patients")
public class PatientController {

    @Autowired
    private PatientService service;

    @PostMapping
    public Patient createPatient(@RequestBody
Patient patient) {
        return service.createPatient(patient);
    }

    @GetMapping
    public List<Patient> getAllPatients() {
        return service.getAllPatients();
    }

    @GetMapping("/{id}")
    public Patient getPatient(@PathVariable Long
id) {
        return service.getPatient(id);
    }

    @PutMapping("/{id}")
    public Patient updatePatient(@PathVariable
Long id, @RequestBody Patient patient) {
        return service.updatePatient(id, patient);
    }

    @DeleteMapping("/{id}")
    public void deletePatient(@PathVariable
Long id) {
        service.deletePatient(id);
    }
}
```

PatientRepository.java:

```
package com.healthhub.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.healthhub.model.Patient;

public interface PatientRepository extends JpaRepository<Patient, Long> {

}
```

PatientService.java:

```
package com.healthhub.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.healthhub.model.Patient;
import com.healthhub.repository.PatientRepository;

@Service
public class PatientService {

    @Autowired
    private PatientRepository repository;

    public Patient createPatient(Patient patient) {
        return repository.save(patient);
    }

    public List<Patient> getAllPatients() {
        return repository.findAll();
    }

    public Patient getPatient(Long id) {
```

```
        return repository.findById(id)
                .orElseThrow(() -> new RuntimeException("Patient not found"));
    }

    public Patient updatePatient(Long id, Patient newPatient) {

        Patient patient = getPatient(id);

        patient.setName(newPatient.getName());
        patient.setEmail(newPatient.getEmail());
        patient.setAge(newPatient.getAge());

        return repository.save(patient);
    }

    public void deletePatient(Long id) {
        repository.deleteById(id);
    }
}
```

Output:

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

healthhub > **add_patient**

Save | Share

POST http://localhost:8080/api/patients **Send**

Docs | Params | Authorization | Headers 8 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

```
{
  "name": "Ashutosh Yadav",
  "email": "john@gmail.com",
  "age": 20
}
```

Body | Cookies | Headers 5 | Test Results | 200 OK • 15 ms • 230 B | Save Response

JSON | Preview | Visualize

```
{
    "id": 2,
    "name": "Ashutosh Yadav",
    "email": "john@gmail.com",
    "age": 20
}
```

healthhub > **patient_id**

Save | Share

GET http://localhost:8080/api/patients/2 **Send**

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results | 200 OK • 10 ms • 230 B | Save Response

JSON | Preview | Visualize

```
{
    "id": 2,
    "name": "Ashutosh Yadav",
    "email": "john@gmail.com",
    "age": 20
}
```

CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

healthhub > **delete_patient**

Save | Share

DELETE http://localhost:8080/api/patients/1 Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Tests | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body | Cookies | Headers 5 | Test Results

200 OK • 47 ms • 184 B • Save Response

Raw | Preview | Visualize

```
Deleted successfully
```

healthhub > **update_patient**

Save | Share

PUT http://localhost:8080/api/patients/2 Send

Docs | Params | Authorization | Headers 8 | Body | Scripts | Tests | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON

Schema | Beautify

```
{
  "name": "name Updated",
  "email": "johnupdated@gmail.com",
  "age": 22
}
```

Body | Cookies | Headers 5 | Test Results

200 OK • 14 ms • 235 B • Save Response

JSON | Preview | Visualize

```
{
    "id": 2,
    "name": "name Updated",
    "email": "johnupdated@gmail.com",
    "age": 22
}
```

Learning Outcomes:

- **Develop a Spring Boot application**
  Create and run a basic Spring Boot project and understand its role in modern backend development.
- **Apply Inversion of Control (IoC) and Dependency Injection (DI)**
  Implement IoC and DI concepts using Spring annotations such as @Autowired, @Service, and @Repository to manage dependencies efficiently.
- **Design applications using Layered Architecture**
  Structure backend applications into Controller, Service, and Repository layers, ensuring proper separation of concerns and maintainable code.
- **Create RESTful APIs with CRUD functionality**
  Develop REST APIs using appropriate HTTP methods (GET, POST, PUT, DELETE) to perform Create, Read, Update, and Delete operations.
- **Implement DTO-based data transfer**
  Use Data Transfer Objects (DTOs) to separate internal entity models from API request and response structures.